

Conception d'algorithmes corrects et efficaces (R3.02)

Christian Attiogbé

Octobre-novembre 2022 (Version 0, en évolution)

BUT2 - Informatique

Contents

1	Introduction	3
1.1	Bien concevoir les algorithmes	4
1.2	Outils d'analyse des algorithmes	5
2	Rappels : Notions de base de construction d'algorithmes	9
2.1	Des exigences (<i>requirements</i>) au logiciel/système	9
2.2	Caractérisation d'un logiciel ou d'un système	10
2.3	Propriétés (en Logique)	11
2.4	Préconditions, postconditions	11
2.5	Propriétés invariantes	12
2.6	Types	13
2.7	Techniques de raisonnement	13
2.8	Espace, Temps	15
2.9	Structures de données	16
2.10	Conclusion : conception et complexité	17
3	Spécification et construction de programmes	19
3.1	Spécifications	19
3.2	Triplet de Hoare	20
3.3	Terminaison	20
3.4	Construction inductive des algorithmes et programmes . . .	21
3.5	Construction des itérations/boucles	22
4	Methodes algorithmiques	25

4.1	Méthode : Diviser pour régner	25
4.2	Méthode : Algorithmes gloutons	25

Le cadrage du programme national (PN) du BUT Info.

Ressource R3.02 : Développement efficace

Compétences ciblées :

- Développer — c'est-à-dire concevoir, coder, tester et intégrer — une solution informatique pour un client.
- Proposer des applications informatiques optimisées en fonction de critères spécifiques : temps d'exécution, précision, consommation de ressources.. Descriptif : L'objectif de cette ressource est de renforcer l'apprentissage de l'algorithmique afin d'amener vers une efficacité de développement.

Savoirs de référence étudiés

- Développement de structures de données complexes (par ex. : collections, arbres, dictionnaires...)
- Premières approches de l'analyse de la performance (profiling, optimisation, greencode...)

Prolongements suggérés

- Notions de programmation fonctionnelle intégrée à des langages non fonctionnels (lambda-expressions, clôtures...)
- Appréhension des conséquences d'une faille dans le code

Apprentissages critiques ciblés :

- AC21.03 : Adopter de bonnes pratiques de conception et de programmation
- AC22.01 : Choisir des structures de données complexes adaptées au problème
- AC22.02 : Utiliser des techniques algorithmiques adaptées pour des problèmes complexes (par ex. recherche opérationnelle, méthodes arborescentes, optimisation globale, intelligence artificielle...)

- AC22.04 : Évaluer l'impact environnemental et sociétal des solutions proposées

Mots clés : Structure de données – Performance

Chapter 1

Introduction

Nous nous concentrons sur la construction correctes d'**algorithmes séquentiels**.

Préambule

Définition 1 *Efficace (du Larousse) :*

(1) *Se dit d'un produit, d'une méthode, d'un appareil, etc., qui produisent l'effet attendu ; bon pour.*

(2) *Qui remplit bien sa tâche, qui atteint son but, qui aboutit à des résultats utiles*

L'activité de *construction de logiciels* (appelée aussi *développement* de logiciels) nécessite des méthodes rigoureuses pour assurer de bonnes propriétés aux logiciels produits.

Cette activité de construction de logiciels inclut différentes étapes similaires à celles de construction d'autres produits : analyse, conception, développement, tests, maintenance.

Cet enseignement concerne plus l'étape de conception, charnière entre l'analyse et le développement ; elle en conditionne les résultats.

C'est dans ce contexte que nous allons aborder la conception d'algorithmes corrects et efficaces, au service de la construction rigoureuse (efficace ?) des logiciels.

Ce qu'il faut envisager lorsqu'on construit

Hypothèse : pour une instruction/opération s'exécutant en $1\mu\text{s}$ (10^{-6}s).

taille ↓	complexité				
	$\log_2(n)$	n	$n\log n$	n^2	2^n
n=10	$3\mu\text{s}$	$10\mu\text{s}$	$30\mu\text{s}$	$100\mu\text{s}$	$1000\mu\text{s}$
n=100	$7\mu\text{s}$	$100\mu\text{s}$	$700\mu\text{s}$	1/100s	10^{14} siècles
n=1000	$10\mu\text{s}$	$1000\mu\text{s}$	1/100 s	1s	astronomique*
n=10000	$13\mu\text{s}$	1/100 s	1/7 s	1,7 min	astronomique
n=100000	$17\mu\text{s}$	1/10 s	2 s	2,8h	astronomique

Table 1.1: Temps d'exécution en fonction des coûts et taille des données

(*) astronomique = 1 milliard de milliard d'années ! (10^{18} d'années)

Un programme en n^2 mettra 2,8h pour traiter une donnée de taille 100 000

La population française est estimée à 67 500 000 de personnes ; admettons que la sécurité sociale effectue un traitement qui est en complexité n^2 .

Avec l'hypothèse d' $1\mu\text{s}$ pour une opération élémentaire, il faudrait environ 1890 heures, soit environ 79 jours pour que le traitement soit fini.

Le même traitement en complexité linéaire de n , prendrait un peu plus d'une minutes !

1.1 Bien concevoir les algorithmes

Les *algorithmes* sont à la base des *logiciels* (ou applications logicielles) et des systèmes intégrant des logiciels. Les caractéristiques telles que la correction, la qualité, la performance, le coût, etc des applications dépendent en partie de leur *bonne conception* et à un certain niveau de précision, des algorithmes utilisés dans différentes parties des logiciels. Ces algorithmes comme les logiciels qu'ils engendrent ont des coûts calculables, mesurables ou estimables. Ces coûts permettent également de comparer les algorithmes/logiciels en terme de performance par exemple.

En fonction des critères choisis par le programmeur, on peut calculer ou estimer le coût d'un programme/algorithme.

Généralement on considère le coût en temps, le coût en mémoire (et leur impact sur l'environnement : énergie consommée par les processeurs, le réseau), mais on peut quantifier d'autres coûts selon les circonstances et les projets.

Par exemple, on peut compter :

- les opérations élémentaires : affectation de valeur à une variable, comparaison d'éléments, ...
- les opérations non élémentaires : les accès à la mémoire, les entrées/sorties vers des fichiers ou périphériques, les accès réseau, etc

A partir des temps consommés par les opérations, on estime/calcule le coût en temps de l'algorithme/programme en fonction de la quantité de ces opérations effectuées.

De la même façon, à partir des structures de données utilisées, on estime/calcule l'espace mémoire occupé ou utilisé par un algorithme/programme. Par exemple, les structures de données statiques (Tableaux) et dynamiques (listes, arbres, graphes) n'engendrent pas les mêmes coûts, et ne satisfont pas les mêmes besoins pour les programmeurs.

Pour estimer et analyser les coûts, on utilise les **ordres de grandeur** (cf. cours).

Nous étudierons d'une part les outils d'analyse des algorithmes et d'autre part des méthodes de construction correcte des algorithmes et logiciels.

1.2 Outils d'analyse des algorithmes

Un algorithme résout un problème.

Définition 2 *Algorithme* Un algorithme est une méthode de résolution systématique ou de traitement systématique d'un problème donné ou de réalisation d'un calcul, par une suite d'actions organisées par des structures dites de contrôle, en s'appuyant éventuellement sur des données d'entrée.

Il y a des problèmes qui n'ont pas de solution algorithmique.

Exemple : le problème de Euler (recherche de chemin) ; pas de solution !

Il y a un risque à ne pas savoir identifier les natures des problèmes qu'on a à traiter.

Plusieurs méthodes, techniques, outils, disciplines existent pour cela.

Analyse de la complexité

L'**analyse de la complexité** fournit des outils et des mesures pour comparer des algorithmes du point de vue de leur efficacité ou performance. C'est cela que nous allons étudier dans cet enseignement, en même temps que la conception rigoureuse des algorithmes et donc des programmes ou logiciels ou applications (logicielles).

Il ne faut pas confondre l'analyse de la complexité avec la **théorie de la complexité** qui s'attache elle à la calculabilité, et l'étude des classes de problèmes, **solubles ou non solubles**...

Complexité - (Analyse de la complexité)

L'étude de la complexité fournit des outils et des mesures pour comparer des algorithmes du point de vue de leur efficacité.

Calculabilité - (Théorie de la complexité)

L'étude de la calculabilité permet d'identifier des classes de problèmes ou d'objets mathématiques pour lesquels il existe des solutions algorithmiques. En effet tous les problèmes ne peuvent pas être résolus algorithmiquement.

Ordres de grandeur

On compare les algorithmes (et aussi les programmes dérivés), en calculant la quantité de ressources temps ou espaces mémoires utilisées. Ces quantités considérées sont les ordres de grandeur (au lieu des valeurs exactes). On les estime en prenant en compte par exemple les cas extrêmes (favorable, défavorable, pratique).

Formellement le coût d'un algorithme est donné par une fonction $f(n)$.

Avec les ordres de grandeur on majore, ou on encadre $f(n)$ par d'autres fonctions dont les comportements/croissances sont connus.

Par exemple, le coût d'un algorithme en fonction de la taille des données

n , peut être donné par une fonction $f(n) = n(n+1)/2$

On va donc positionner $f(n)$ par rapport à des fonctions connues au lieu de réétudier complètement $f(n)$

Définition 3 *Ordres de grandeur* Un ordre de grandeur est un **ensemble de fonctions** (ou une classe), utilisé comme référence pour apprécier la croissance d'une fonction de coût : n^2 , $\log_2(n)$, n^p , etc

$$O(g) = \{ h \in \mathbb{N} \rightarrow \mathbb{R}_+ \mid \exists (c, n_0). (c \in \mathbb{R}, n_0 \in \mathbb{N} \wedge \forall n. ((n \in \mathbb{N} \wedge n \geq n_0) \Rightarrow h(n) \leq c.g(n))) \}$$

Ainsi, on exprime que le coût $f(n)$ d'un algorithme est dans $O(g(n))$

Voici quelques ordres de grandeur des mesures de complexité:

Classe	dénomination ou type de complexité
$O(n!)$	complexité factorielle ; complètement démentiel
$O(2^n)$	complexité exponentielle ; ces algorithmes ont un coût catastrophique quand la taille n des données grandit.
$O(n^2)$	complexité quadratique ; de nombreux algorithmes ; acceptables pour des tailles assez grandes des données (> 10-100taines milliers).
$O(n \log n)$	complexité pseudo-linéaire.
$O(n)$	complexité linéaire (liée à la taille n des données),
$O(\log n)$	complexité logarithmique (en base 2).
$O(1)$	en temps constant, indépendamment des entrées de l'algo.

On compare les algorithmes sur la base de leur complexité. La performance ou l'efficacité d'un algorithme sont des notions liées à la classe de complexité de l'algorithme.

Terminologie

- complexité spatiale : coût en quantité d'espace mémoire
- complexité temporelle : coût en rapidité d'exécution

- **grand O (O)** : comportement asymptotique (taille des données très grande, aux abords de l'infini). O donne un ensemble de majorants d'une fonction.
thêta (Θ) : comportement dans les cas pratiques ; Θ donne une mesure plus juste intégrant minorants et majorants.
petit o (o) : comportement dans les cas favorables ; donne les minorants.

Pour aller plus loin

des ouvrages et liens d'entrée sur de nombreuses ressources pédagogiques :

https://fr.wikipedia.org/wiki/Complexit%C3%A9_en_temps

https://en.wikipedia.org/wiki/Time_complexity#Table_of_common_time_complexities

Chapter 2

Rappels : Notions de base de construction d'algorithmes

2.1 Des exigences (*requirements*) au logiciel/système

Définition 4 *Exigence (wikipedia).* Une exigence est, dans le domaine de l'ingénierie, un besoin, une nécessité, une attente auquel un produit ou un service doit répondre ou une contrainte qu'il doit satisfaire. L'exigence peut être exprimée par une partie prenante (utilisateur, client, commercial, analyste de marchés, gestionnaire de produits, etc.) ou déterminée par les processus d'ingénierie et en particulier les activités d'études.

Définition 5 *Software requirements (wikipedia).* Software requirements for a system are the description of what the system should do, the service or services that it provides and the constraints on its operation.

Plusieurs étapes sont nécessaires pour passer des exigences au logiciel (le produit) ; on le construit progressivement à travers des phases ou étapes d'**analyse**, de **conception** et de **développement**.

En phase de conception, on conçoit les traitements à réaliser via des **méthodes**, **algorithmes** et des **structures les données** appropriées.

Définition 6 *Algorithme* Un algorithme est une méthode de résolution systématique ou de traitement systématique d'un problème donné ou de réalisation d'un calcul, par une suite d'actions organisées par des structures dites de contrôle, en s'appuyant éventuellement sur des données d'entrée.

Définition 7 *Programme* Un programme est la traduction d'un algorithme dans un langage de programmation donné, en vue de sa compilation en code exécutable, et son exécution sur un/des processeur/s (ou machines).

Pour un même algorithme, on peut ainsi avoir des programmes très différents en fonction des langages et des environnements de programmation. Il est difficile de comparer des programmes. De même, l'exécution d'un programme va dépendre des environnements (compilateurs, systèmes d'exploitation, processeurs) dans lequel on l'effectue.

En conséquence l'algorithme reste la référence pour l'étude des solutions (algorithmiques) lorsqu'on veut les comparer. D'autres critères de performance des environnements d'exécution peuvent être utilisés pour effectuer des comparaisons à ce niveau.

Définition 8 *Système* Un système est un ensemble de composants interagissant pour effectuer une ou des tâche spécifiques pour lesquelles le système est dédié. Les composants peuvent être de différente nature : des logiciels, des matériels, des humains.

2.2 Caractérisation d'un logiciel ou d'un système

Le fonctionnement d'un système se manifeste par une succession d'états. De la même manière le fonctionnement d'un logiciel se manifeste par une succession d'états.

L'état d'une entité est sa perception à un instant précis dans le temps. Une entité peut changer d'état d'un instant à un autre ; elle évolue (dans le temps).

Ainsi un algorithme, lors de son fonctionnement, évolue dans le temps en changeant d'états ; il passe d'un état à un autre sous l'effet de ses instructions. Une instruction peut donc modifier l'état d'un algorithme. L'état d'un algorithme est perçu à travers les valeurs de ses variables. Les valeurs de chacune des ses variables constitue l'état de l'algorithme à un moment donné.

Définition 9 *Etat* L'état d'un système est sa perception à un instant donné, à travers des valeurs ou des données qui le caractérisent.

Définition 10 *Espace d'états* L'espace d'états d'un système est l'ensemble des états par lequel il peut passer pendant son fonctionnement.

2.3 Propriétés (en Logique)

Une propriété est une caractéristique que possède une entité (un objet, un humain, ...).

Une propriété est formellement exprimée avec le langage de la Logique (des prédicats, premier ordre, etc).

Des exemples de propriété :

- le tableau est blanc ;
- le ciel est jaune ;
- tout processus a u identifiant unique ;
- le processus est bloqué ;
- tous les oiseaux à bec plat ne sont pas des cigognes ;
- tout dossier doit avoir au moins deux référents ;
- les indices du tableau doivent être situés entre les valeurs b_i et b_s ;
- le canal est crypté ;
- les processus doivent s'exécuter en exclusion mutuelle ;
- le serveur doit supporter la charge de 1000 connexions/min ;
- tout entier naturel a un successeur.

Une propriété est donc un énoncé qui peut être vrai ou faux : une proposition, un prédicat.

On a appris à formaliser des énoncés en Logique (du premier ordre) à l'aide de propositions. et de prédicats

2.4 Préconditions, postconditions

Une précondition est une propriété qui doit être vraie avant d'effectuer une action, afin qu'elle puisse se dérouler convenablement.

Par exemple la précondition pour une action de division d'un entier n par un autre entier m , est que m soit non nulle, car la division n'est pas définie sinon.

La précondition pour allumer une lampe, un appareil, une machine, est qu'il y ait de l'énergie.

Une postcondition est une propriété vraie après le déroulement d'une action (ou d'une suite d'actions).

Par exemple après avoir casser un oeuf +++

Les actions ici sont par exemples des instructions d'un programme, ou le déroulement d'un algorithme, un traitement, ou leurs abstractions.

Des exemples

$$\{n \in \mathbb{N} \wedge m \in \mathbb{N} \wedge m \neq 0\} \quad \mathbf{q, r = n/m} \quad \{q \in \mathbb{N} \wedge r \in \mathbb{N} \wedge n = m * q + r\}$$

$$\{energy > 10\} \quad \mathbf{activate_wifi()} \quad \{wifi = on\}$$

$$\{True\} \quad \mathbf{If (k > g) Then m \leftarrow k Else m \leftarrow g Fi} \quad \{m = max(\{k, g\})\}$$

$$\{False\} \quad \mathbf{pm \leftarrow ppcm(k, g)} \quad \{\exists u. (u \in \mathbb{N}_1 \wedge pm = u \times k)\}$$

2.5 Propriétés invariantes

Une propriété invariante d'un système est une propriété toujours vraie pour le système pendant son fonctionnement.

Des exemples de propriété sont :

- Tout réel a toujours un réel supérieur.

$$\forall r. (r \in \mathbb{R} \Rightarrow (\exists s. (s \in \mathbb{R} \wedge s > r)))$$

- Les animaux respirent

$$\forall ani. (ani \in ANIMAL \Rightarrow respire(ani))$$

- Tout triangle a trois côtés

$$\forall f. (f \in FIGURE \wedge f \in TRIANGLE \Rightarrow nbCotes(f) = 3)$$

- Il n'y a pas de variables sans type

$$\forall va.(va \in VARIABLE \Rightarrow (\exists ty.(ty \in TYPE \wedge (va, ty) \in typeOf)))$$

- Tout processeur a une unité arithmétique et logique
- ...

Dans l'analyse ou la modélisation d'un énoncé pour concevoir un algorithme ou un logiciel, on a besoin d'exprimer des invariants.

2.6 Types

Les types sont des catégories de données ayant les mêmes propriétés, ou un ensemble de données de même nature.

Nous avons utilisé en programmation, les types comme : entier, réel, structure composée d'un entier, d'un autre entier et d'un réel, tableau de type, etc

Un type élémentaire est un ensemble d'éléments. On construit des types structurés en composant des types, élémentaires ou non.

Les types de bases sont des ensembles : $\mathbb{N}$ (NAT), $\mathbb{Z}$ (INTEGER), $\mathbb{R}$ (REAL), etc

On utilise les types pour structurer les données.

En Logique on définit un ensemble par une propriété ; par conséquent les types sont aussi vus comme des propriétés.

Un entier paire, est un entier qui a la propriété d'être divisible par 2. Les entiers paires constituent un sous-ensemble des entiers ayant la propriété d'être divisible par 2. Soit $\mathbb{P} \subseteq \mathbb{N}$ l'ensemble des entiers paires.

Dire qu'un entier n donné est paire revient à exprimer :

$$n \in \mathbb{P}$$

2.7 Techniques de raisonnement

Pour analyser les besoins et construire des programmes efficaces, quelques techniques de raisonnement sont nécessaires. Nous en rappelons brièvement ici quelques unes.

Récurrence. Le raisonnement par récurrence permet de généraliser une démonstration d'une propriété sur un ensemble d'objets (de même nature) sans avoir à le faire sur tous les objets.

Par exemple, démontrer une propriété sur les entiers naturels.

Exemple : démontrer que

$$S_n = \sum_{k=0}^n k = 0 + 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Pour cela une démonstration par récurrence vous évite de vérifier la propriété pour tout $n \in \mathbb{N}$. Ainsi le schéma de la démonstration consiste à :

- **(dé)montrer/prouver** la propriété pour le premier objet (de rang 0, cas de base, initialisation), ou les premiers objets,
- **supposer** que la propriété est vraie pour le $n^{ième}$ objet (de rang n , c'est l'étape dite hérédité), puis sur cette base
- **(dé)montrer/prouver** que, elle est vraie pour le $n + 1^{ième}$ objet. En fait, on montre que la propriété vraie à l'ordre n , implique la propriété vraie à l'ordre $(n+1)$.

Illustration.

$$S_n = \sum_{k=0}^n k = 0 + 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

$$0 = \frac{0(0+1)}{2}$$

$$1 = 0 + 1 = \frac{1(1+1)}{2}$$

...

supposons

$$S_n = \frac{n(n+1)}{2}$$

qu'en est-il de S_{n+1} ?

$$\begin{aligned} S_{n+1} &= 0 + 1 + 2 + \dots + n + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) \end{aligned}$$

$$\begin{aligned}
 &= \frac{n(n+1)}{2} + \frac{2(n+1)}{2} \\
 &= \frac{(n+1)(n+2)}{2}
 \end{aligned}$$

en conclusion on a :

$$\forall n \in \mathbb{N}. S_n = \sum_{k=0}^n k = \frac{n(n+1)}{2}$$

La méthode se généralise au delà des entiers (sur un ensemble d'éléments). On peut l'appliquer à n'importe quel ensemble (bien fondé : muni d'un ordre entre les éléments) pour faire des démonstrations.

Induction (du particulier au général). L'induction est la méthode de raisonnement qui consiste à généraliser des lois ou des propriétés sur des objets à partir de leur observation sur des faits ou des objets particuliers.

L'induction (Logique, Informatique), est souvent confondue avec la récurrence. Preuve par induction ou preuve par récurrence.

Dédution (du général au particulier). C'est la méthode de raisonnement qui consiste à attribuer une propriété à un (ou des objets) particuliers en partant du fait que la propriété est générale à l'ensemble dont est issu l'objet (ou les objets).

Par exemple, sachant que *un nombre entier est divisible par 9 si la somme de ses chiffres est un multiple de 9*, si on arrive à montrer que la somme des chiffres d'un entier N donné est multiple de 9, alors on déduit que N est divisible par 9.

↪ Comparer les deux algorithmes !

L'induction et la déduction sont deux méthodes ou outils efficaces d'analyse de problème et de conception d'algorithmes.

2.8 Espace, Temps

Les programmes ou logiciels engendrés par les algorithmes consomment des ressources comme le temps et la mémoire, mais aussi de l'énergie.

L'espace et le temps sont deux paramètres très utilisés pour étudier, mesurer, comparer les programmes. Les programmes étant très dépendants de leur environnement (langages, processeurs, réseaux), on étudie plutôt ces paramètres sur les algorithmes et non directement sur les programmes directement.

Dans la suite, du cours le paramètre **temps** évoque le temps/durée (estimé) consommé par un algorithme pour effectuer un traitement jusqu'à sa terminaison. Le temps est mesuré en fonction des temps estimés des opérations et des quantités effectuées de ces opérations.

De même, l'espace mémoire est celui (estimé) consommé par un algorithme pour effectuer un traitement jusqu'à sa terminaison. L'espace mémoire est mesuré en fonction des structures de données et de leur nombre, alloué pour toute le déroulement d'un algorithme.

Ces paramètres, on parle aussi de ces ressources ou de ces grandeurs, existent en grand nombre mais ne sont pas illimité.

Un algorithme sera plus ou moins coûteux en temps ou en mémoire. C'est un critère d'appréciation et de catégorisation des algorithmes.

2.9 Structures de données

Une structure de données est un moyen d'organiser et de représenter des données en vue de simplifier leur manipulation.

Une structure de données permet de manipuler aisément les données. Il existe différentes structures de données telles que tuple, liste, tableau, file, ensemble, sac, arbre, graphe, etc.

Ces structures de données sont employées en fonction de leur adéquation ou efficacité à la résolution du problème qu'on a sous la main.

Le choix d'une structure de donnée relève de la conception et ne doit pas précéder l'analyse d'un problème.

Des données déjà structurées sont parfois une entrée d'un problème ; il convient d'en tenir compte mais, cela n'est pas un carcan pour la conception car les données peuvent toujours être pré-traitées ou post-traitées. L'analyse globale des coûts de toutes les étapes permet de décider des bons choix.

Par exemple, il peut être parfois plus intéressant (= moins coûteux) de pré-traiter des données puis utiliser des méthodes algorithmiques éprouvées, plutôt que de garder les structures des données en entrée et de devoir

concevoir un algorithme très coûteux en ressource (temps, espace).

Le choix d'une ou des structures de données pour un algorithme ou pour un projet, doit être basé sur des critères soigneusement établis, et leurs coûts étudiés.

2.10 Conclusion : conception et complexité

La prise en compte de l'étude de la complexité impacte la conception des algorithmes en particulier et du logiciel de façon générale. Inversement, une conception rigoureuse des algorithmes impacte leur correction et leur complexité.

Fort heureusement, en algorithmique, on dispose de méthodes et d'outils bien établis pour aider le concepteur-programmeur dans son travail.

Chapter 3

Spécification et construction de programmes

On construit les programmes à partir de leur spécification issue de l'analyse des besoins ou des problème à résoudre. Les méthodes de construction rigoureuse existent pour guider les différentes étapes ; la Logique de Hoare permet d'écrire des spécifications et de raisonner sur ces spécifications et les programmes qu'elles permettent de construire.

3.1 Spécifications

Le couple (précondition, postcondition) est appelé la *spécification* d'un algorithme ou d'un programme.

$$\text{spécification} \equiv (\text{precondition}, \text{postcondition})$$

On parle aussi de **contrat** ; la précondition est la partie du contrat honorée par l'utilisateur du programme alors que la postcondition est la partie du contrat honorée par le programme.

On construit un programme à partir de sa *spécification*, de telle sorte que le programme satisfasse la spécification : étant donné la précondition, le programme établit lorsqu'il est terminé, la postcondition.

La terminaison du programme reste aussi à établir.

3.2 Triplet de Hoare

La logique de HOARE est un outil formel qui permet d'effectuer des preuves de corrections partielles des programmes.

Le triplet de Hoare exprime la spécification de la correction partielle d'un programme.

Définition 11 *Triplet de Hoare* Un triplet de Hoare est une expression de la forme

$$\{P\} S \{Q\}$$

où S est un programme et P et Q sont des formules logiques.

Définition 12 *Triplet valide* Le triplet $\{P\} S \{Q\}$ est valide (i.e. toujours vrai) si et seulement si toute exécution finie de S commençant par un état initial qui rend vraie la formule $\{P\}$ se termine par un état final qui vérifie la formule $\{Q\}$.

La démonstration de la validité de $\{P\} S \{Q\}$ constitue la preuve de correction du programme S . Plus généralement, la logique de Hoare, ramène toutes les démonstrations à la preuve de la validité d'une suite d'expressions de la forme $\{P\} S \{Q\}$.

Cet type de preuve s'effectue en utilisant des règles précises de déduction qui sont notées comme suit :

$$\frac{\{P\} S \{Q\}}{\{P'\} S' \{Q'\}}$$

Cette notation se lit : si $\{P\} S \{Q\}$ est valide alors $\{P'\} S' \{Q'\}$ l'est aussi.

Tout programme structuré est obtenu en composant des opérations élémentaires et des structures de contrôle. Pour l'opération élémentaire qu'est l'affectation, et pour chacune des structures de contrôle de base que sont la séquence, la conditionnelle "Si Alors Sinon", la répétitive "Tantque", nous disposons de règles précises de déduction. Le respect de ces règles est le garant d'une preuve correcte.

Les règles concernant les autres structures de contrôle se déduisent des équivalences entre les structures.

3.3 Terminaison

La validité du triplet de Hoare $\{P\} S \{Q\}$ ne garantit en rien que le programme S termine, c'est à dire, atteint la fin d'exécution (un état final). Il

se peut que le programme s'exécute indéfiniment. En effet, pour prouver la correction partielle d'un programme nous avons supposé la terminaison de ce programme, mais il ne s'agissait que d'une hypothèse de travail.

En excluant la récursivité et le branchement inconditionnel (instruction goto qui n'a pas sa place dans le contexte des algorithmes structurés) le non arrêt d'un programme ne peut provenir que d'une répétitive. Prouver la terminaison d'un programme revient donc à prouver que chaque boucle du programme ne peut s'exécuter qu'un nombre fini de fois.

Pour prouver la terminaison d'une boucle il faut exhiber une série ou suite s , fonction des variables intervenant dans la boucle et vérifiant les propriétés suivantes :

- s prend ses valeurs sur un domaine scalaire muni d'une relation d'ordre ;
- s est strictement monotone (i.e. strictement croissante ou strictement décroissante) ;
- s est majorée si elle est croissante, minorée si elle est décroissante.

Généralement, ce sont des suites à valeurs dans $\mathbb{N}$, décroissantes et minorées par zéro qui s'imposent naturellement.

3.4 Construction inductive des algorithmes et programmes

Dans un contexte de programme séquentiel, de style impératif, un programme est construit à l'aide d'instructions élémentaires et d'instructions composées d'autres instructions à l'aide de structures dites de contrôle. Les programmes ont ainsi une structure bien précise, qui guide leur construction.

Principales structures de contrôles des programmes :

- Affectation : instruction élémentaire permettant d'affecter une valeur ou une expression à une variable.
- Séquence : il s'agit d'un enchaînement (à l'aide d'un opérateur de séquence, souvent noté ";") de deux instructions élémentaires ou non. On peut ainsi enchaîner une longue suite d'instructions.

- **Conditionnelle** : il s'agit de faire dépendre l'exécution d'une suite d'instructions T, d'une condition C (c'est un prédicat). Si la condition C est vraie la suite d'actions T est exécutée. On peut prévoir ou non une suite d'action T2 à exécuter lorsque la condition n'est pas vraie. Ainsi, l'opérateur de composition des instructions prend trois arguments Si(Cond, T1, T2). Pratiquement, on utilise la forme Si C alors T1 sinon T2 fin.
- **Itération** : il s'agit ici de répéter ou itérer une suite d'actions T aussi longtemps qu'une certaine condition donnée/voulue C est vraie. La valeur de la condition dépend en partie des variables utilisées dans T. L'opérateur prend la forme Tantque C faire T fin.

Parmi ces instructions élémentaires et structures de contrôle, la construction des itérations mérite une attention particulière.

3.5 Construction des itérations/boucles

Construire rationnellement un algorithme ou un programme consiste à le voir comme un mécanisme faisant passer un "système" d'une situation initiale appelée précondition à une situation finale nommée "postcondition" ou "but".

On construit un programme à partir de sa *spécification* : (precondition, postcondition).

La construction de boucle est systématisée par les constituants suivants :

1. **Invariant** (de boucle) : un prédicat représentant une propriété caractéristique du problème traité.
2. **Condition d'arrêt** : C'est également un prédicat, qui exprime la condition d'arrêt de la boucle
3. **Progression** : c'est une action/instruction ou un bloc d'actions.
4. **Initialisation** : c'est une action/instruction ou un bloc d'actions, effectué une seule fois au début.
5. **Expression de Terminaison** : c'est une expression à valeur entière (qui va croître tout le temps ou bien décroître tout le temps).

Les relations suivantes réunissent ces constituants :

R1 La conjonction de l'invariant et de la condition d'arrêt conduit au but recherché par le programme :

$(\text{Invariant et condition d'arrêt}) \Rightarrow \text{post-condition}$

R2 La progression doit

- (a) conserver l'invariant. Avant l'exécution de la progression, le prédicat invariant et non condition d'arrêt est vrai. Avant et après l'exécution de la progression, l'invariant doit être vrai.
- (b) faire décroître strictement l'expression de terminaison. Ainsi à chaque tour de boucle, l'expression est positive ou nulle, et strictement inférieure à sa valeur précédente ; la condition d'arrêt sera donc atteinte au bout d'un temps fini. Cela permet de garantir que le programme construit *se termine* (donc pas de boucles infinies).

R3 L'initialisation doit instaurer l'invariant. Notons que la pré-condition de l'initialisation est la pré-condition du programme et sa post-condition est l'invariant de la boucle.

A travers les relations **R1**, **R2**, **R3**, on note que l'invariant est un constituant fondamental pour la conception/construction des boucles. Ainsi il est raisonnable de commencer la construction de boucle par l'identification de l'invariant.

Chapter 4

Methodes algorithmiques

en chantier

4.1 Méthode : Diviser pour régner

4.2 Méthode : Algorithmes gloutons